SafeLeWM

Helping a frozen world model finish tasks safely

The aim: reach the original task goal while staying clear of hazards, without retraining LeWM.

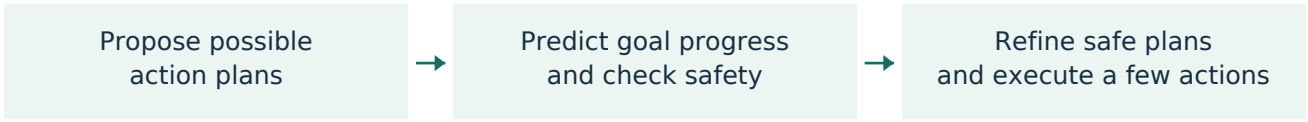
1. Add a safety requirement to the task

LeWM is a world model: it predicts how a scene might change under possible actions. A planner uses those predictions to choose what to do. We keep the model's weights frozen and introduce virtual keep-out regions into its tasks. A route that reaches the goal can now be unacceptable because it crosses a hazard.

In our current PushT setup, the pusher and T-block must reach their recorded goal positions, with the T-block at the desired orientation. The full pusher disk must keep an extra distance from hazards, called the safety margin. The T-block is excluded from the hazard rule. The zones do not physically block motion, and are supplied separately to the checker rather than painted into LeWM's observations.

2. Use SafeCEM to search for safe progress

CEM, the cross-entropy method, searches by sampling action plans, scoring them and repeatedly refining the promising ones. SafeCEM adds a safety check: **plans that pass the check take priority, then LeWM's goal score chooses among them.** It executes only the first few actions, observes the result and replans. Our current runs execute five actions per replan.



Observe what happened, then plan again.

LeWM supplies the predicted task progress. For PushT, the safety checker uses the current pusher position and velocity, its known motion dynamics and the hazard geometry. If no checked plan is available, the controller stops and the episode is unfinished. Stopping safely is not counted as completing the task.

3. Why the default settings can struggle

The planner has a limited search budget. Once hazards rule out direct routes, many candidate plans may be rejected, and the remaining choices may make little progress. Settings that were useful for the original task can therefore leave SafeCEM safe but unfinished. Tuning changes how it spends its search effort so that useful safe alternatives are easier to find.

What stays fixed: the learned model, the task goal and the requested safety margin. The parameters being adjusted belong to the planner.

Why tune, and what do we know?

The main knobs control how far the planner looks, where it searches and how much useful search history it keeps.

Lookahead. A shorter horizon means fewer action variables to search and can reduce accumulated error in the model's task predictions. Replanning lets short plans build into a longer maneuver. A longer horizon can still be valuable when useful progress requires looking further ahead.

Sampling spread. Broader exploration can reveal routes around a hazard. Too little spread can keep proposing similar blocked plans; too much can waste samples on unsafe or unhelpful actions.

Linking action blocks (temporal correlation). Related changes across successive action blocks encourage more coherent candidate plans.

Keeping good plans (candidate retention). Reusing promising safe plans between search iterations and replans reduces the need to rediscover them.

A useful tradeoff: in the earlier PushT study, shortening lookahead from 25 to 10 actions allowed 750 candidate plans instead of 300 per iteration with roughly the same amount of model prediction work. The planner could look less far ahead while considering more alternatives.

What the experiments support

Earlier fixed-preset tuning improved safe completion on reserved cases: PushT at 12 px rose from **2/36 to 19/36**, and Cube at 1 cm from **0/36 to 12/36**. Those studies used their older layouts and safety definitions. Several settings changed together, so these results do not isolate which individual change caused the improvement.

The latest PushT smoke test used one scene, one seed and a manually chosen preset. Island, rounded-bar and wide-gate runs completed in **26, 40 and 40 actions**, maintaining 6 px clearance around the full pusher at recorded physics steps. The shared unchecked reference finished in 17 actions but crossed the hazards. It used the same preset, so these videos demonstrate safety checking rather than a new default-versus-tuned comparison.

The local problem: choose settings automatically

Different environments can favor different search tradeoffs. We want one reusable configurator that tries bounded settings on development tasks, prioritizes safe completion and then efficiency, and freezes the best evaluated configuration for testing on fresh cases. It may return different settings for PushT, TwoRoom and Cube. The safety margin stays fixed.

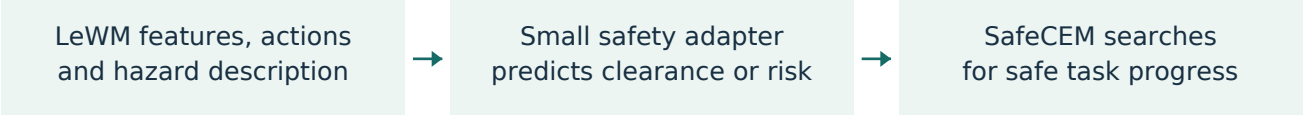
Random search, Bayesian optimization and evolution are possible outer tuning methods. The output can initially be a fixed configuration; an execution-time adaptive controller is a separate extension. No automatic tuning comparison has yet been run on the new three-shape setup.

Evidence: Principia, task 18; configs/lewm18/protocol.json; progress/receipts/20261029-pusht-hazard-smoke.json. View the private smoke-test videos and records.

A safety adapter for frozen LeWM

PROPOSED EXTENSION / NOT YET IMPLEMENTED OR VALIDATED

The aim is to replace environment-specific predictive checkers with a small learned safety adapter. It would translate LeWM's features and candidate actions into a safety estimate for supplied hazards. **Only the adapter would learn; LeWM's original weights would stay frozen.**



LeWM supplies the goal score. The adapter supplies the safety estimate.

Inputs and output. The hazard description specifies the region, the protected body or point, and the required margin. The adapter would predict the minimum clearance during an action chunk, or its violation risk. Checking only the endpoint could miss a hazard crossed along the way.

Learn from the evidence we already have

Recorded images can be encoded by frozen LeWM, and saved executed states can supply clearance and violation labels. Because our hazards are virtual, one recorded trajectory can be labelled against many hazard placements without new simulator rollouts. Training and test data must be separated by original trajectory before relabelling.

Keep the adapter and tuner separate

The **safety adapter** learns to assess proposed behavior. The **automatic configurator** chooses SafeCEM's search settings using actual safe-completion outcomes. Initially, the same training procedure could fit a separate small adapter for each environment, since different LeWM checkpoints need not share aligned latent features. The requested safety margin would remain fixed during tuning.

The intended experience: supply the hazards, automatically calibrate the adapter, tune the planner, then run. We would still provide the hazard meaning and reliable labels for actual behavior.

The first test is reliable safety prediction

Latent goal distance is not automatically physical hazard clearance. Earlier probes showed that accurate readouts of real observations can still fail on imagined futures. This adapter must therefore be checked against actual outcomes on held-out trajectories and plans of the kind CEM proposes. Conservative error allowances would need calibration; a safety prediction alone is not a guarantee. We would measure completion alongside violations so that excessive caution is not mistaken for success.

Related reading: AnySafe (2025) studies latent safety filters conditioned on user-specified constraints. It motivates this direction but does not validate the proposed LeWM adapter.